

CHAPTER 2:

Jakarta Servlet, Servlet Life cycle, Sharing Information

JAKARTA EE

A solid green horizontal bar spanning the width of the slide, located at the bottom.

CONTENT

2.1. Jakarta Servlet

2.2. Asynchronous Processing

2.3. Contexts and Dependency Injection (CDI)

2.4. REST API

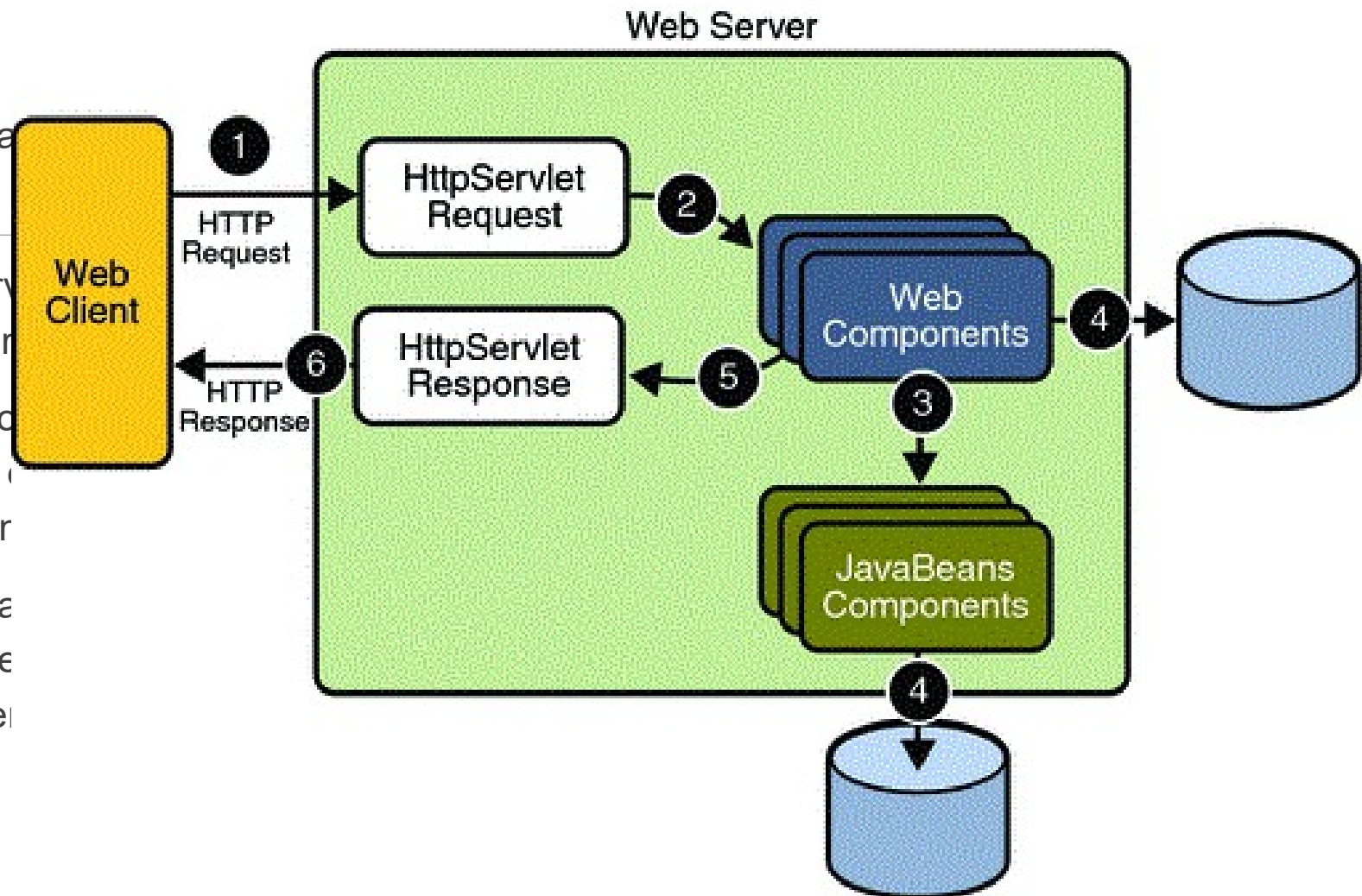
2.5. Web Socket

What is a

A serv
gener

Like d
Java
dynar

Conta
servle
imple



3

Definitions

A servlet is a Java class that can be loaded dynamically into and run by a special Web server.

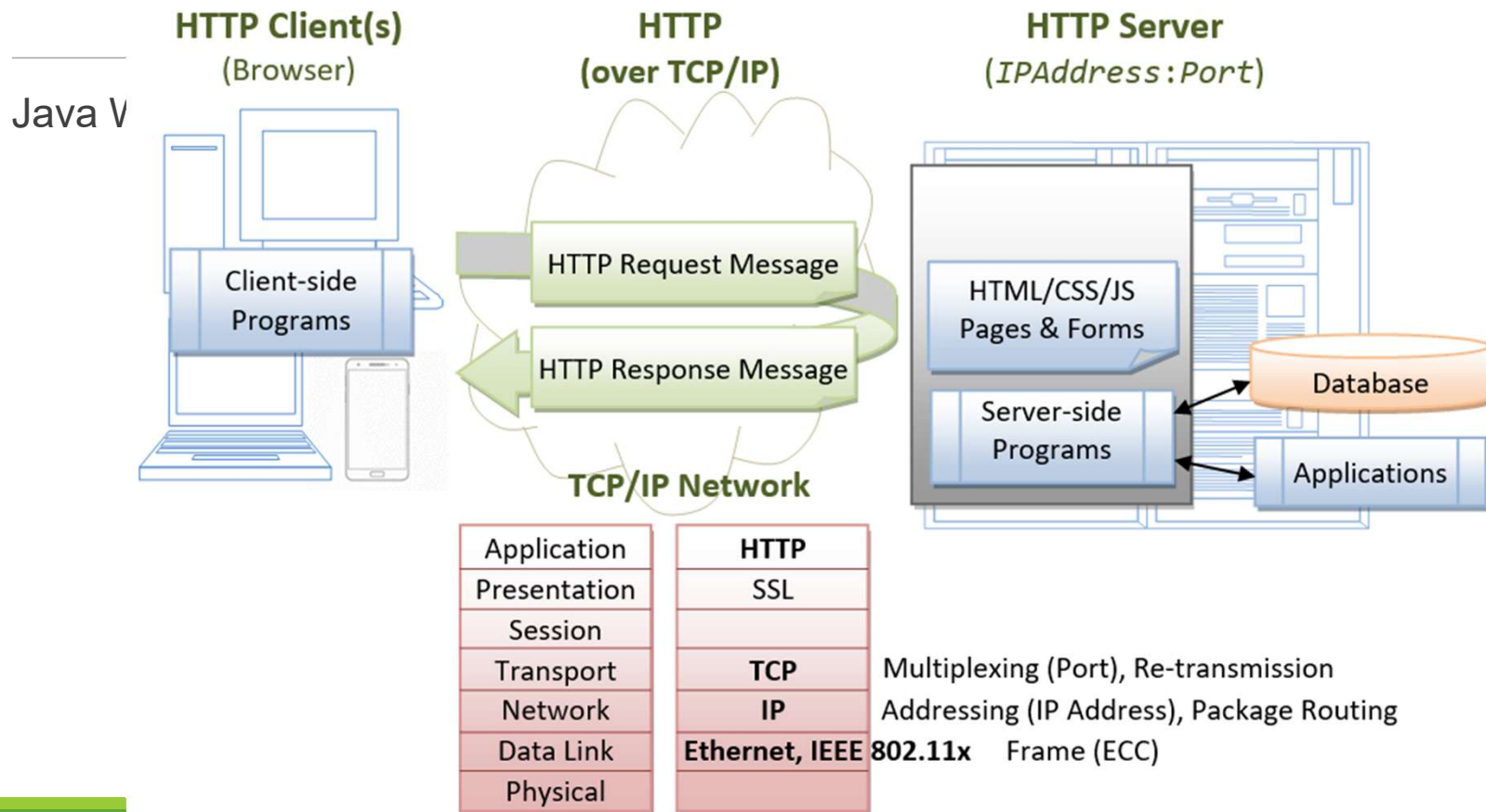
The **servlet-aware Web server**, is known as **servlet container**.

Servlets interact with clients via a **request-response** model based on HTTP.

The **servlet container** must support **HTTP** as the protocol for **client requests** and **server responses**.

A servlet container also can support similar protocols such as HTTPS (HTTP over SSL) for secure transactions

Servlet Request/Response Handling

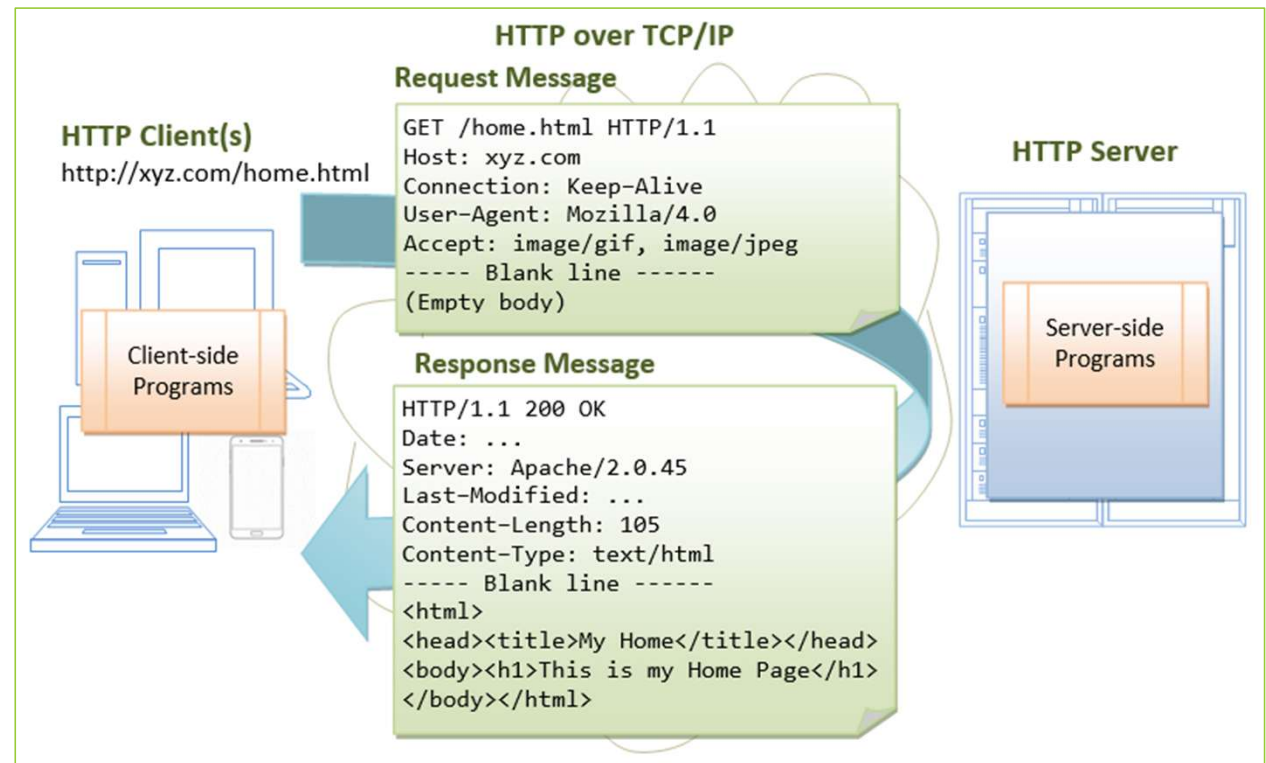
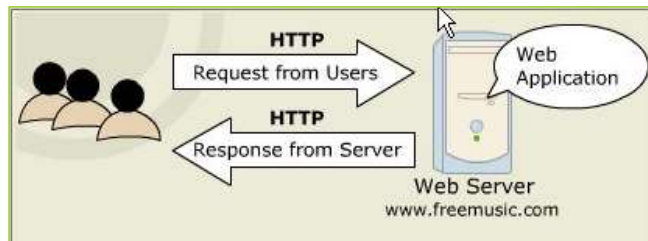


Execution of Servlets

1. The clients send the request to the Web server.
2. The Web server receives the request.
3. The Web server passes the request to the corresponding Servlet.
4. The Servlet read:
 - The explicit data, this includes an HTML form on a Web page or a custom HTTP client program.
 - The implicit HTTP request data, this includes cookies, media types and compression schemes the browser understands.
5. The Servlet processes the data and generate the results (the response on the form of output).
6. The Servlet send the response back to the Web server.
7. The Web server sends the response back to the Client and the Client Browser displays it on the screen.

Servlet Communication & Protocols

Message



Servlet Packages

1. ***jakarta.servlet***: contains a number of classes and interfaces that describe and define the contracts between a **servlet class** and the **runtime environment** provided for an instance of such a class by a conforming **servlet container**.
2. ***jakarta.servlet.http***: contains a number of classes and interfaces that describe and define the contracts between a **servlet class** running under the **HTTP protocol** and the **runtime environment** provided for an instance of such a class by a conforming servlet container.
3. ***jakarta.servlet.annotation***: contains annotations to annotate servlets, filters, and listeners. It also specifies metadata for annotated components.
4. ***jakarta.servlet.descriptor***: contains types that provide programmatic access to a web application's configuration information.

jakarta.servlet Packages

Interfaces	Classes
Servlet ServletConfig ServletContext ServletRequest ServletResponse RequestDispatcher Filter FilterChain FilterConfig ServletRequestListener ServletRequestAttributeListener ServletContextListener ServletContextAttributeListener	GenericServlet ServletRequestWrapper ServletResponseWrapper ServletInputStream ServletOutputStream ServletContextEvent ServletContextAttributeEvent ServletRequestEvent ServletRequestAttributeEvent ServletException UnavailableException

Servlet vs GenericServlet vs HttpServlet

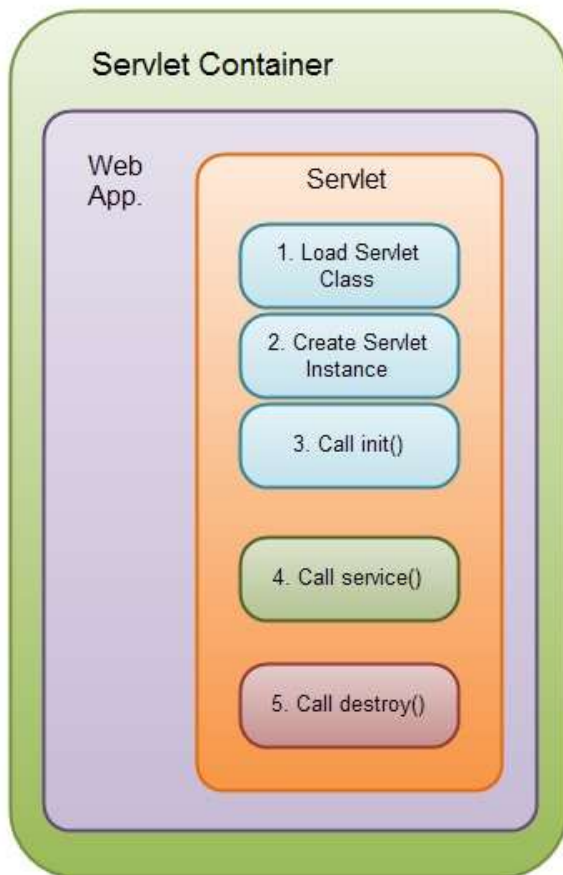
- 1. *jakarta.servlet.Servlet*:** is the top level interface in the hierarchy of Java servlets which defines all the necessary methods to be implemented by the servlets.
- 2. *jakarta.servlet.GenericServlet*:** is an abstract class which implements `jakarta.servlet.Servlet` interface and provides methods to write protocol independent servlets.
- 3. *jakarta.servlet.http.HttpServlet*:** is also an abstract class which extends `jakarta.servlet.GenericServlet` and provides methods to write HTTP-specific servlets.

	Servlet	GenericServlet	HttpServlet
What is it?	Interface	Abstract Class	Abstract Class
Package	jakarta.servlet	jakarta.servlet	jakarta.servlet.http
Hierarchy	Top level interface	Implements Servlet Interface	Extends GenerticServlet
Methods	init() service() destroy() getServletConfig() getServletInfo()	init() service() destroy() getServletConfig() getServletInfo() log() getInitParameter() getInitParameterNames() getServletContext() getServletName()	doGet() doPost() doPut() doDelete() doHead() doOptions() doTrace() getLastModified() service()
Abstract Methods	All methods are abstract	Only service() method is abstract	No abstract method
When to use	Use it when we want to develop own Servlet container	Use to write protocal independent servlets	Use to write HTTP-specific servlets

Http methods and Servlet methods

Http method	Description	Servlet Method
GET	The GET method requests a representation of the specified resource. Requests using GET should only retrieve data.	doGet
POST	The POST method submits an entity to the specified resource, often causing a change in state or side effects on the server.	doPost
PUT	The PUT method replaces all current representations of the target resource with the request payload.	doPut
DELETE	The DELETE method deletes the specified resource.	doDelete
HEAD	The HEAD method asks for a response identical to a GET request, but without the response body.	doHead
OPTIONS	The OPTIONS method describes the communication options for the target resource.	doOptions
TRACE	The TRACE method performs a message loop-back test along the path to the target resource.	doTrace
CONNECT	The CONNECT method establishes a tunnel to the server identified by the target resource.	
PATCH	The PATCH method applies partial modifications to a resource.	

Servlet Lifecycle



There are 5 steps:

1. Load **Servlet** Class.
2. Create Instance of **Servlet**.
3. Call the servlets **init()** method.
4. Call the servlets **service()** method.
5. Call the servlets **destroy()** method.

Step 1, 2 and 3 are executed **only once**, when the servlet is initially loaded

Step 4 is executed **multiple times** - once for every HTTP request to the servlet.

Step 5 is executed when the servlet container **unloads** the servlet.

Servlet Lifecycle

init, service, and destroy are lifecycle methods

1) void init(ServletConfig config): called when servlet is instantiated; return before any other methods will be called.

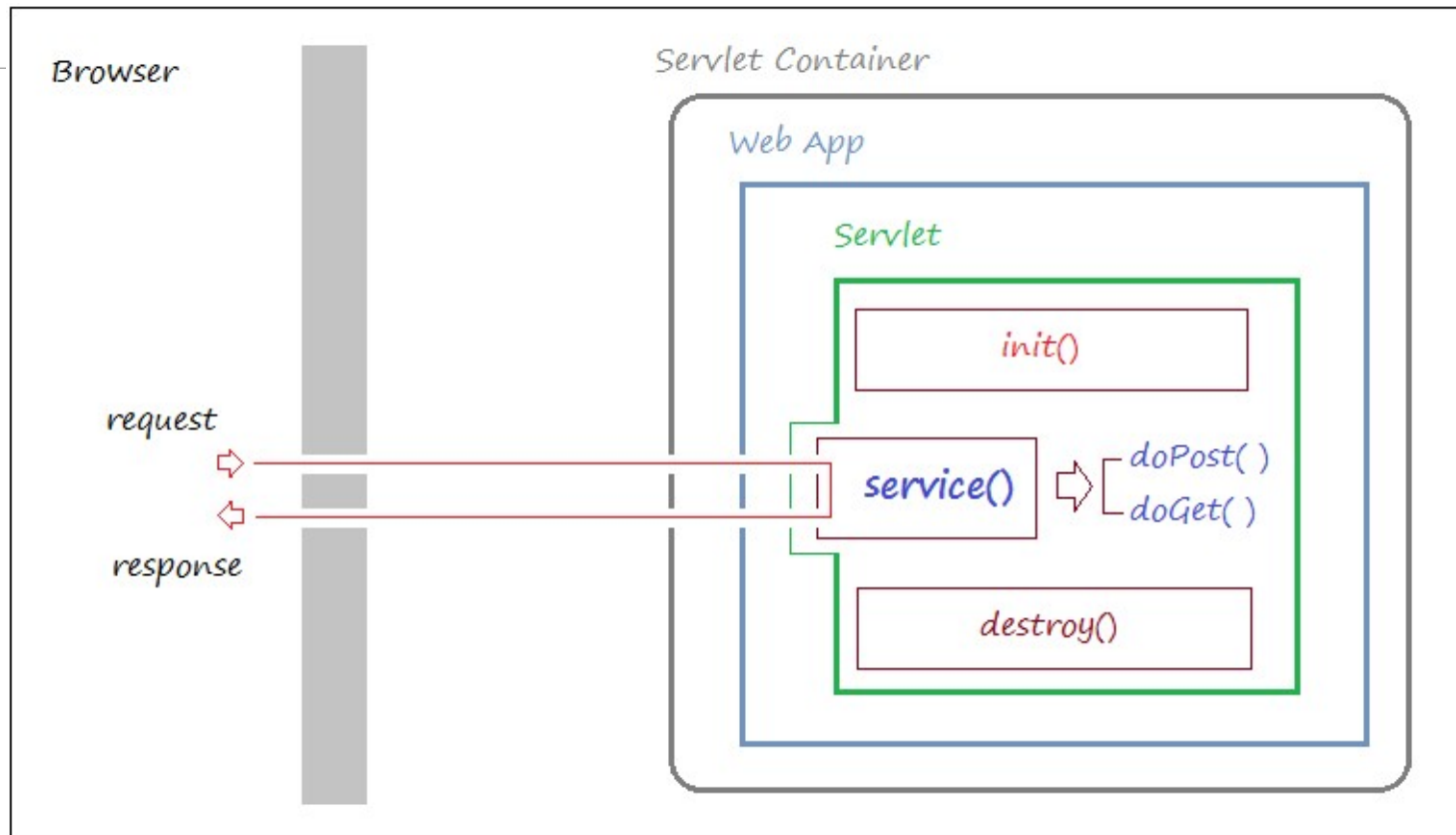
2) void service(ServletRequest req, ServletResponse res): method called directly by server when an HTTP request is received.

3) void destroy(): called when server shuts down. Cleans up whatever resources are being held (e.g., memory, file handles, threads) and makes sure that any persistent state is synchronized with the servlet's current in-memory state.

4) ServletConfig getServletConfig(): returns a servlet config object, which contains any initialization parameters and startup configuration for this servlet.

5) String getServletInfo(): returns a string containing information about the servlet, such as its author, version and copyright.

Servlet Lifecycle



Anatomy of a Servlet

1. HttpServletRequest object

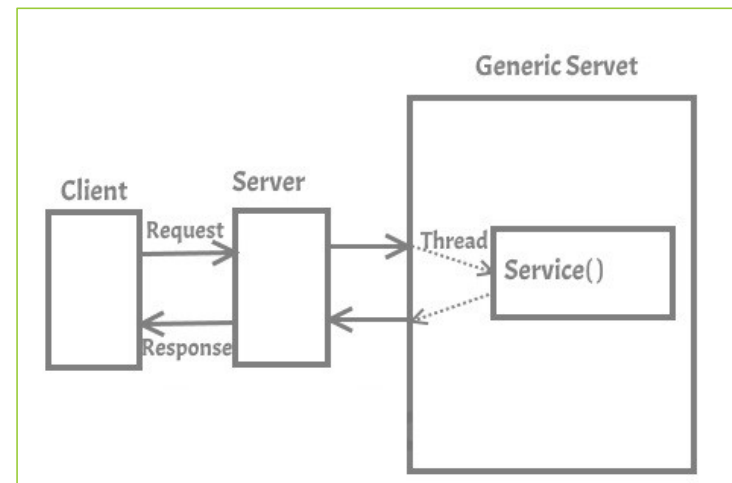
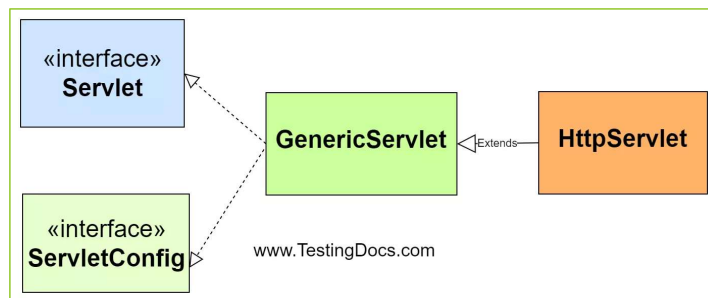
- Information about an HTTP request
- Headers
- Query String
- Session
- Cookies

2. HttpServletResponse object

- Used for formatting an HTTP response
- Headers
- Status codes
- Cookies

Types of Servlet

The jakarta.servlet.GenericServlet

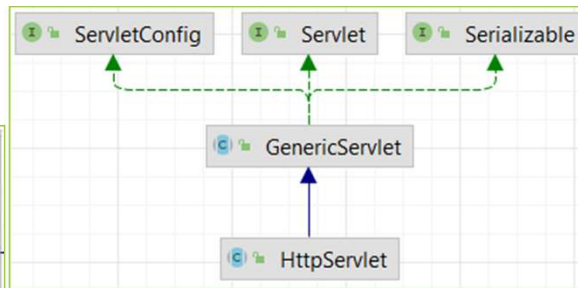


Example: <http://localhost:8080/hello/mygenericservlet> (helloWorld)

Types of Servlet

jakarta.servlet.http.HttpServlet

HttpServlet		
getModifiedSince(HttpServletRequest)	long	
doTrace(HttpServletRequest, HttpServletResponse)	void	
maybeSetLastModified(HttpServletResponse, long)	void	
init(ServletConfig)	void	
getMethodNotSupportedCode(String)	int	
doPost(HttpServletRequest, HttpServletResponse)	void	
doOptions(HttpServletRequest, HttpServletResponse)	void	
getAllDeclaredMethods(Class<HttpServlet>)	Method[]	
doHead(HttpServletRequest, HttpServletResponse)	void	
doPut(HttpServletRequest, HttpServletResponse)	void	
doGet(HttpServletRequest, HttpServletResponse)	void	
doDelete(HttpServletRequest, HttpServletResponse)	void	
service(HttpServletRequest, HttpServletResponse)	void	
service(ServletRequest, ServletResponse)	void	



Hello World!

[Hello Servlet](#)
[My Generic](#)


```
<html>
<body>
<br/>
<a href="hello-servlet">Hello Servlet</a>
<br/>
<a href="mygenericservlet">My Generic</a>
<br/>

<form action="greeting" method="get">
  <input type="text" name="name"/>
  <input type="submit" value="Submit"/>
</form>

</body>
</html>
```

HTTPServlet

When using **HttpServlet**, you will work with the **HttpServletRequest** and **HttpServletResponse** objects that represent the **servlet request** and the **servlet response**, respectively

HttpServlet overrides the service method in **GenericServlet** and adds **another service method**

The **new service** method in **HttpServlet** then examines the **HTTP method used to send the request** (by calling `request.getMethod()`) and call one of the following methods: **doGet**, **doPost**, **doHead**, **doPut**, **doTrace**, **doOptions**, and **doDelete**

We rarely need to override the **service methods** anymore. Instead, you override **doGet** or **doPost** or both **doGet** and **doPost**

HTTPServlet - Methods

- 1) void **doGet** (HttpServletRequest request, HttpServletResponse response):
handles GET requests
- 2) void **doPost** (HttpServletRequest request, HttpServletResponse response):
handles POST requests
- 3) void **doPut** (HttpServletRequest request, HttpServletResponse response):
handles PUT requests
- 4) void **doDelete** (HttpServletRequest request, HttpServletResponse response):
handles DELETE requests
- 5) void **doHead** (HttpServletRequest request, HttpServletResponse response) handles the HEAD request

HTTPServlet - Methods

6) void **doOptions** (HttpServletRequest request, HttpServletResponse response) handles the OPTIONS request

7) void **doTrace** (HttpServletRequest request, HttpServletResponse response) handles the TRACE request.

8) void **service** (ServletRequest request, ServletResponse response) dispatches the request to the protected service method by converting the request and response object into http type.

9) void **service** (HttpServletRequest request, HttpServletResponse response) receives the request from the service method, and dispatches the request to the doXXX() method depending on the incoming http request type.

HttpServletRequest Request

HttpServletRequest represents the servlet request in the **HTTP environment**

- 1) **java.lang.String getContextPath()**: returns the portion of the request URI that indicates the context of the request.
- 2) **Cookie[] getCookies()**: returns an array of **Cookie** objects.
- 3) **java.lang.String getHeader(java.lang.String name)**: returns the value of the specified HTTP header.
- 4) **java.lang.String getMethod()**: returns the name of the HTTP method with which this request was made.
- 5) **java.lang.String getQueryString()**: returns the query string in the request URL.

HttpServletRequest Request

6) HttpSession getSession(): returns the session object associated with this request. If none is found, creates a new session object.

7) HttpSession getSession(boolean create): returns the current session object associated with this request. If none is found and the create argument is **true**, create a new session object.

HTTPServlet Response

HttpServletResponse represents the **servlet response** in the **HTTP environment**.

Here are some of the methods defined in it:

- 1) **void addCookie(Cookie cookie)**: adds a cookie to this response object.
- 2) **void addHeader(java.lang.String name, java.lang.String value)**: adds a header to this response object.
- 3) **void sendRedirect(java.lang.String location)**: sends a response code that redirects the browser to the specified location

HTML Forms

- A web application almost always contains **one or more HTML forms** to **take user input**. We can easily send an HTML form from a servlet to the browser. When the user submits the form, **values entered in the form are sent to the server as request**
- The value of an **HTML input field** (a text field, a hidden field, or a password field) or **textarea** is sent to the server **as a string**. An **empty input field or text area** sends an **empty string**. As such, **ServletRequest.getParameter** that takes an **input field name** **never returns null**
- An HTML **select element** also sends a string to the server. If none of the options in the select element is selected, the **value of the option that is displayed is sent**
- A **multiple-value select element** sends a **string array** and has to be handled by **ServletRequest.getParameterValues**

HTML Forms

- A **checked checkbox** sends the string “on” to the server. An **unchecked checkbox** sends **nothing to the server** and `ServletRequest.getParameter(fieldName)` **returns null**
- **Radio buttons** send the value of **the selected button** to the server. If none of the buttons is selected, **nothing is sent to the server** and `ServletRequest.getParameter(fieldName)` **returns null**
- If a form contains **multiple input elements with the same name**, **all values will be submitted** and we have to use `ServletRequest.getParameterValues` to retrieve them. `ServletRequest.getParameter` will only return the **last value**

Example: `FormUploadServlet`

Servlet Mapping

1. Map a servlet with an annotation - `@WebServlet`
2. Map a servlet with Deployment Descriptor (web.xml)

Map a servlet with an annotation - `@WebServlet`

With `@WebServlet` we can specify:

- **Name:** defines the name of servlet
- **urlPatterns:** maps the servlet to the pattern
- **loadOnStartup:** loads the servlet at the time of deployment or server start if value is positive
- **Description:** description about servlet
- **initParams:** takes multiple `@WebInitParam` annotation

Map a servlet with an annotation - @WebServlet

```
@WebServlet(name="/WelcomeServlet",urlPatterns="/WelcomeServlet",loadOnStartup=1, description="Welcome Servlet")
```

```
@WebServlet(name="/WelcomeServlet",urlPatterns={"/WelcomeServlet",  
"/HelloServlet"}, description="Welcome Servlet")
```

```
@WebServlet (urlPatterns= {"/emailList", "/email/*"})
```

Map a servlet with an annotation - @WebServlet

@WebInitParam enables us to configure one init param and provides name , value and description attribute.

- **Name:** name of init param
- **Value:** value of init param
- **Description:** description of init param

```
@WebServlet(name="/WelcomeServlet",urlPatterns={"/WelcomeServlet"},
initParams={
    @WebInitParam(name="Param1",value="Param 1 Value",description="param1"),
    @WebInitParam(name="Param2",value="Param 2 Value",description="param1")
})
```

Map a servlet with Deployment Descriptor (web.xml)

XML elements for working with **servlet** mapping:

- **<servlet-name>**: Specifies a unique name for the servlet that's used to identify the servlet within the web.xml file. This element is required for both the **servlet element** and the **servlet mapping element**
- **<servlet-class>**: Specifies the class for the servlet. Note that this element **includes the package and name for the class but not the .class extension.**
- **<url-pattern>**: Specifies the URL or URLs that are mapped to the specified servlet. This pattern **must begin with a front slash.**

Map a servlet with Deployment Descriptor (web.xml)

```
<servlet>
  <servlet-name>myGenericServlet</servlet-name>
  <servlet-class>com.fit.se.helloworld.myGenericServlet</servlet-class>
  <init-param>
    <param-name>Greetings</param-name>
    <param-value>Hello</param-value>
  </init-param>
</servlet>
<servlet-mapping>
  <servlet-name>myGenericServlet</servlet-name>
  <url-pattern>/mygenericservlet</url-pattern>
</servlet-mapping>
<context-param>
  <param-name>name</param-name>
  <param-value>Thu Ha</param-value>
</context-param>
```


@WebFilter Annotation

WebFilter is used to define a filter in a web application. Filters are used to perform tasks on either the request to a resource, the response from a resource, or both.

The @WebFilter annotation provides several attributes for configuring filters:

- `filterName`: The name of the filter.
- `urlPatterns`: Specifies the URL patterns to which the filter applies.
- `servletNames`: Specifies the servlet names to which the filter applies.
- `dispatcherTypes`: Specifies the dispatcher types (REQUEST, FORWARD, INCLUDE, ERROR, ASYNC).
- `initParams`: Specifies the initialization parameters for the filter.

@WebFilter Annotation (cont.)

```
@WebFilter(urlPatterns = {"/admin/*", "/user/*", "/login"},
           initParams = {
@WebInitParam(name = "param1", value = "value1"),
@WebInitParam(name = "param2", value = "value2")
           }
)
public class AuthenticationFilter implements Filter {
// Filter implementation
}
```

@WebFilter Annotation (cont.)

Example:

- **Tạo 3 Filter** (AuthFilter, EncodingFilter, FileLogFilter)
- **Tạo LoginServlet và LogoutServlet**
- **JSP**: login.jsp, home.jsp, secure/secret.jsp
- webapp **secure/secret.jsp**

WebFilter – web.xml

```
<filter>
  <display-name>authentication</display-name>
  <filter-name>authentication</filter-name>
  <filter-class>fit.iuh.edu.servlet.AuthenticationFilter</filter-class>
  <init-param>
    <param-name>username</param-name>
    <param-value>test</param-value>
  </init-param>
  <init-param>
    <param-name>password</param-name>
    <param-value>123456</param-value>
  </init-param>
</filter>
<filter-mapping>
  <filter-name>authentication</filter-name>
  <url-pattern>/login</url-pattern>
</filter-mapping>
```

@MultipartConfig Annotation

MultipartConfig is an annotation in **Jakarta Servlet** used to tell the server that this servlet will handle **multipart/form-data** requests — most commonly for **file uploads** from HTML forms.

When an HTML form uses `enctype="multipart/form-data"` (required for uploading files), a servlet will not parse the file data by default.

To enable file upload handling:

- Add `@MultipartConfig` to your servlet.
- Use `request.getPart(...)` or `request.getParts()` to access the uploaded files and other form fields

@MultipartConfig Annotation

The @MultipartConfig annotation provides several attributes for configuring file uploads:

- location: Specifies the directory location where files will be stored.
- fileSizeThreshold: Specifies the size threshold after which the file will be written to disk.
- maxFileSize: Specifies the maximum size allowed for uploaded files.
- maxRequestSize: Specifies the maximum size allowed for multipart/form-data requests.

@MultipartConfig Annotation

Syntax:

```
@WebServlet("/upload")
@MultipartConfig(location = "/tmp",
fileSizeThreshold = 1024 * 1024 * 1, // 1MB
maxFileSize = 1024 * 1024 * 10, // 10MB
maxRequestSize = 1024 * 1024 * 15 // 15MB
)
public class FileUploadServlet extends HttpServlet {
// Servlet implementation
}
```

Example: Upload File

MultipartConfig – web.xml

```
<servlet>
  <description></description>
  <display-name>fileUpload</display-name>
  <servlet-name>fileUpload</servlet-name>
  <servlet-class>fit.iuh.edu.servlet.FileUploadServlet</servlet-class>
  <multipart-config>
    <location>/tmp</location>
    <max-file-size>20848820</max-file-size>
    <max-request-size>418018841</max-request-size>
    <file-size-threshold>1048576</file-size-threshold>
  </multipart-config>
</servlet>
```


Handling Servlet Lifecycle Events

Servlet Lifecycle Events		
Object	Event	Listener Interface and Event Class
Web context	Initialization and destruction	<code>jakarta.servlet.ServletContextListener</code> and <code>ServletContextEvent</code>
Web context	Attribute added, removed, or replaced	<code>jakarta.servlet.ServletContextAttributeListener</code> and <code>ServletContextAttributeEvent</code>
Session	Creation, invalidation, activation, passivation, and timeout	<code>jakarta.servlet.http.HttpSessionListener</code> , <code>jakarta.servlet.http.HttpSessionActivationListener</code> , and <code>HttpSessionEvent</code>
Session	Attribute added, removed, or replaced	<code>jakarta.servlet.http.HttpSessionAttributeListener</code> and <code>HttpSessionBindingEvent</code>
Request	A servlet request has started being processed by web components	<code>jakarta.servlet.ServletRequestListener</code> and <code>ServletRequestEvent</code>
Request	Attribute added, removed, or replaced	<code>jakarta.servlet.ServletRequestAttributeListener</code> and <code>ServletRequestAttributeEvent</code>

You can monitor and react to events in a servlet's lifecycle by defining listener objects whose methods get invoked when lifecycle events occur.

```
@WebListener()  
public class SimpleServletListener implements ServletContextListener,  
    ServletContextAttributeListener, HttpSessionListener {
```

Sharing Information

Collaborating web components share information by means of objects that are maintained as attributes of four scope objects. You access these attributes by using the `getAttribute` and `setAttribute` methods of the class representing the scope.

Sharing Information

Scope Object	Class	Accessible From	
Web context	jakarta.servlet.ServletContext	Web components within a web context. See [accessing-the-web-context] .	// set/get application scoped attribute req.getServletContext().setAttribute("name", "application scoped attribute"); String applicationScope = (String) req.getServletContext().getAttribute("a");
Session	jakarta.servlet.http.HttpSession	Web components handling a request that belongs to the session. See [maintaining-client-state] .	// set/get session scoped attribute HttpSession session = req.getSession(); session.setAttribute("name", "session scoped attribute"); String sessionScope = (String) session.getAttribute("b");
Request	Subtype of jakarta.servlet.ServletRequest	Web components handling the request.	// set/get request scoped attribute req.setAttribute("name", "request scoped attribute"); String requestScope = (String) req.getAttribute("c"); // send redirect to other servlet req.getRequestDispatcher("get-attributes").forward(req, resp);
Page	jakarta.servlet.jsp.JspContext	The Jakarta Server Pages page that creates the object.	<% page.setAttribute("name", "page scoped attribute"); String pageStr = page.getAttribute("name").toString(); %>

Sharing Information

In a multithreaded server, shared resources can be accessed concurrently. In addition to scope object attributes, shared resources include in-memory data, such as instance or class variables, and external objects, such as files, database connections, and network connections.

Concurrent access can arise in several situations.

- Multiple web components accessing objects stored in the web context.
- Multiple web components accessing objects stored in a session.
- Multiple threads within a web component accessing instance variables.

Sharing Information

A web container will typically create a thread to handle each request. To ensure that a servlet instance handles only one request at a time, a servlet can implement the `SingleThreadModel` interface. If a servlet implements this interface, no two threads will execute concurrently in the servlet's service method. A web container can implement this guarantee by synchronizing access to a single instance of the servlet or by maintaining a pool of web component instances and dispatching each new request to a free instance. This interface does not prevent synchronization problems that result from web components' accessing shared resources, such as static class variables or external objects.

Asynchronous Processing

Asynchronous Processing in **Jakarta Servlet** is a mechanism that allows a servlet to handle a request **without keeping the HTTP connection blocked until the task is finished**, improving performance for long-running operations such as calling external APIs, large database queries, or file processing.

Web containers in application servers normally use a server thread per client request.

Under heavy load conditions, containers need a large amount of threads to serve all the client requests.

Scalability limitations include running out of memory or exhausting the pool of container threads.

To create scalable web applications, you must ensure that no threads associated with a request are sitting idle, so the container can use them to process new requests.

Asynchronous Processing

There are two common scenarios in which a thread associated with a request can be sitting idle:

- The thread needs to wait for a resource to become available or process data before building the response. For example, an application may need to query a database or access data from a remote web service before generating the response.
- The thread needs to wait for an event before generating the response. For example, an application may have to wait for a Jakarta Messaging message, new information from another client, or new data available in a queue before generating the response.

Asynchronous Processing

Synchronous mode (default):

1. When a client sends a request, a servlet thread starts processing it.
2. The thread **waits** until the task is finished.
3. During that time, the thread is **blocked** and cannot serve other requests.

Asynchronous mode:

1. The servlet thread switches the request to **async mode** (`request.startAsync()`).
2. The original thread is **released** back to the thread pool to handle other requests.
3. When the background task finishes, the servlet sends the response to the client.

Asynchronous Processing

To enable asynchronous processing on a servlet, set the parameter `asyncSupported` to true on the `@WebServlet` annotation as follows:

```
@WebServlet(urlPatterns={"/asyncservlet"}, asyncSupported=true)
public class AsyncServlet extends HttpServlet { ... }
```

The `jakarta.servlet.AsyncContext` class provides the functionality that you need to perform asynchronous processing inside service methods. To obtain an instance of `AsyncContext`, call the `startAsync`

```
public void doGet(HttpServletRequest req, HttpServletResponse resp) {
    ...
    AsyncContext acontext = req.startAsync();
    ...
}
```

The `AsyncListener` class provides the functionality that you can use to listen the states of async thread.

```
public class MyAsyncListener implements AsyncListener {
    //...
}
```

Asynchronous Processing

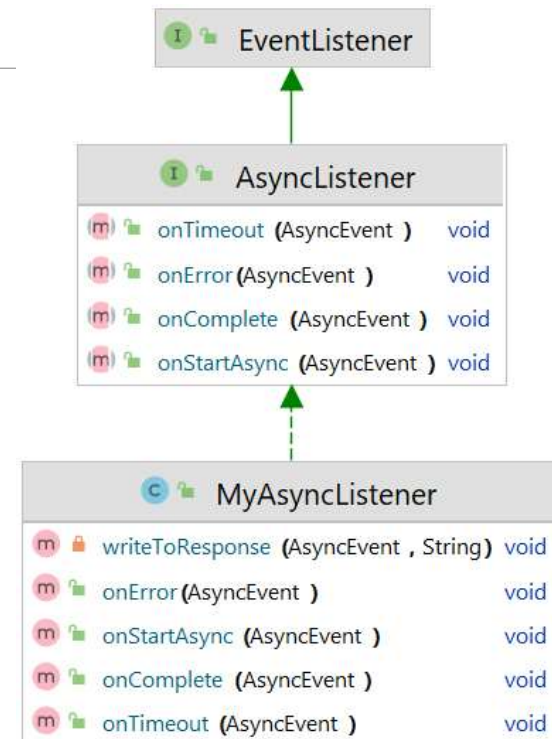
Example

```
@WebServlet(name = "myAsyncServlet", urlPatterns = {"/async-test"},
            asyncSupported = true)
public class MyAsyncServlet extends HttpServlet {
    @Override
    protected void doGet (HttpServletRequest req,
                          HttpServletResponse resp) throws ServletException {

        PrintWriter writer = resp.getWriter();
        AsyncContext asyncContext = req.startAsync();
        asyncContext.addListener(new MyAsyncListener());

        asyncContext.start(new Runnable() {
            @Override
            public void run () {
                String msg = task();
                writer.println(msg);
                asyncContext.complete();
            }
        });
    }

    private String task () {...}
}
```



Read more: <https://github.com/eclipse-ee4j/jakartaee-tutorial/blob/master/src/main/asciidoc/servlets/servlets012.adoc>

Contexts and Dependency Injection (CDI)

Simplifying Dependency Management in Enterprise Applications

- Part of Jakarta EE specification
 - Provides a type-safe, dependency injection framework
 - Integrates with other Jakarta EE technologies (Servlets, JPA, JSF, EJB, etc.)
 - Manages bean lifecycle and contextual scopes automatically
- 

Why CDI?

- Eliminates manual object creation (`new`)
 - Promotes loose coupling and testability
 - Provides contextual lifecycle management
 - Supports interceptors, decorators, and events
- 
- A solid green horizontal bar spanning the width of the slide, located at the bottom.

Contexts and Dependency Injection (CDI)

- **Tomcat 11** operates only as a Servlet container, supporting Servlet and JSP technologies.
- **CDI** (Contexts and Dependency Injection) is part of the Jakarta EE specification and requires an implementation (e.g., Weld)
- To enable CDI on Tomcat, **additional CDI dependencies** must be explicitly included
- Maven add dependency for weld Servlet (implementation of CDI for Tomcat/Jetty)
pom.xml

Contexts and Dependency Injection (CDI)

pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.jboss.weld.servlet</groupId>
    <artifactId>weld-servlet-shaded</artifactId>
    <version>5.1.2.Final</version>
  </dependency>
```

bean.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
       bean-discovery-mode="all">

</beans>
```

Contexts and Dependency Injection (CDI)

Annotation CDI in classes:

`@Named`

`@ApplicationScoped,`

`@SessionScoped,`

`@RequestScope`

`@Qualifier`

`@Produces`

Add Annotation for properties or methods:

`@Inject`

Contexts and Dependency Injection (CDI)

▼ com.fit.democdi

- © GreetingServlet
- © HelloBean
- © HelloServlet
- © ResourceProducer

```
@Named no usages new *
@RequestScoped
public class HelloBean {
    public String sayHello() { no usages new *
        return "Hello from CDI Bean!";
    }
}
```

```
@ApplicationScoped new *
public class ResourceProducer {
    @Produces// tạo ra bean cần dùng bất cứ đâu trong ứng dụng new *
    public String appName() {
        return "Employee Management System";
    }
}
```

```

@WebServlet("/greeting") new *
public class GreetingServlet extends HttpServlet {
    @Inject 1 usage
    private HelloBean helloBean;

    @Inject 1 usage
    private String appName;

    @Override 5 usages new *
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
//        super.doGet(req, resp);
        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println("<h2>RequestScoped: "+helloBean.sayHello()+"</h2>");
        out.println("<h2>ApplicationScoped: "+appName+"<h2>");
    }
}

```

- ✓  cddemo2
 - © Hello
 - © World
 - © XinchaoServlet

Contexts and Dependency Injection (CDI) using

1. Install JAR for CDI in Maven

- **weld-servlet-shaded** (version >5.x, Jakarta EE 10+) (Tomcat do not include CDI)
- **jakarta.enterprise.cdi-api** (version 4.x)
- **jakarta.annotation-api** (version 2.x)

2. Create beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
                           https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"
       bean-discovery-mode="all"
       version="4.0">
</beans>
```

3. Declare Weld Listener web.xml

```
<!-- Weld Servlet Listener -->
<listener>
  <listener-class>org.jboss.weld.environment.servlet.Listener</listener-class>
</listener>

<context-param>
  <param-name>jakarta.enterprise.inject.scan.implicit</param-name>
  <param-value>true</param-value>
</context-param>
```

4. Create CDI code: HelloBean.java

```
@ApplicationScoped no usages
public class HelloBean {
    public String sayHello() { no usages
        return "Hello from CDI on Tomcat 11!";
    }
}
```

5. Create Servlet using Bean

```
@WebServlet("/cdiTest")
public class CdiTestServlet extends HttpServlet {
    // private static final long serialVersionUID = 1L;
    public CdiTestServlet() {} no usages
    @Inject 1 usage
    private HelloBean helloBean;

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println(helloBean.sayHello());
}
```